Reinforcement Learning

Hyoung Joon, Kim

July 11, 2026

Contents

Chapter 1

Introduction

This document is a set of notes of algorithms of reinforcement learning and theories of reinforcement learning. Following books were mainly used for studying algorithms and theories.

- [Sut20];
- [Gra20];
- [Zha25];

Part I

Algorithms

Chapter 2

Value-based Methods

Chapter 3

Policy-based Methods

3.1 Introduction

Policy gradient method, unlike value-based methods, tries to predict the optimal policy itself, instead of obtaining it from value functions. It has many advantages over value-based methods; It is more efficient in handling large state-action spaces, and has stronger generalization abilities.

3.2 REINFORCE

3.2.1 Main Idea

REINFORCE is the most basic form of policy gradient method. It's the vanilla policy gradient method.

3.2.2 Objective Function

REINFORCE uses following objective function;

$$J(\pi_{\boldsymbol{\theta}}) = \mathbb{E}_{\tau\sim\pi_{\boldsymbol{\theta}}}[G_t] = \mathbb{E}_{\tau\sim\pi_{\boldsymbol{\theta}}}\left[\sum_{t=0}^{T-1}\gamma^t r_{t+1}\right]. \tag{3.1}$$

We use some tricks to compute the gradient of $J(\pi_{\boldsymbol{\theta}})$ Then we get

$$\nabla_{\boldsymbol{\theta}} J(\pi_{\boldsymbol{\theta}}) = \mathbb{E}_{\tau\sim\pi_{\boldsymbol{\theta}}}\left[\sum_{t=0}^{T-1} G_t \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t|s_t)\right]. \tag{3.2}$$

Now we can use this to update the parameter $\boldsymbol{\theta}$. Since we cannot compute the exact expectation, we use Monte Carlo sampling to approximate $\nabla_{\boldsymbol{\theta}} J(\pi_{\boldsymbol{\theta}})$ as following:

$$\nabla_{\boldsymbol{\theta}} J(\pi_{\boldsymbol{\theta}}) \approx \sum_{t=0}^{T-1} G_t \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t|s_t) \tag{3.3}$$

3.2.3 Pseudocode

Algorithm 1: REINFORCE

Input : $\boldsymbol{\theta}$ ..... initial parameter
γ ..... discount rate
α ..... learning rate

Output: $\pi_{\boldsymbol{\theta}}$ ..... learned policy

initialize policy $\pi_{\boldsymbol{\theta}}$
repeat
  generate episode $\{s_0, a_0, r_1, \ldots, s_{T-1}, a_{T-1}, r_T\}$ following $\pi_{\boldsymbol{\theta}}$
  for $t = 0, 1, \ldots, T-1$ **do**
    `// compute return-to-go`
    $G_t = \sum_{k=t+1}^{T} \gamma^{k-t-1} r_k$
    `// policy update`
    $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \alpha \nabla_{\boldsymbol{\theta}} \log \pi_{\boldsymbol{\theta}}(a_t|s_t) G_t$
until convergence
return $\pi_{\boldsymbol{\theta}}$

3.2.4 Implementation Notes

- When computing return-to-go, we can iterate backward from the terminal time to start, since $G_t \leftarrow \gamma G_{t+1} + r_{t+1}$ where $t = 0, 1, 2, \ldots, T-1$.

3.2.5 Pros and Cons

Pros:

- No need for model;
- Can cover both discrete and continuous actions;

Cons:

- Large variance;
- May converge to suboptimal policy;

Chapter 4

Model-based Methods

Part II

Theories

Chapter 5

Basic Notations

5.1 Introduction

In this section, we discuss common concepts that are used frequently in reinforcement learning.

5.2 State, Action, Policy and Reward

5.2.1 State

The **state** is the status of the agent with respect to the environment. The set of all possible states, the **state space**, is denoted as $\mathcal{S}$.

5.2.2 Action

The **action** is what agent can do, when a state is given. By choosing the action and taking it, the agent moves to the next state, which is called **state transition**. The set of all possible actions in state s, **action space** for state s, is denoted as $\mathcal{A}(s)$.

5.2.2.1 Transition Probability

Note that the transition can be stochastic. That is, the same choice of action in the same state may not lead to equivalent next state. We denote this probability with conditional porbability as following:

$$p(s'|s,a) \tag{5.1}$$

where $s', s \in \mathcal{S}$ and $a \in \mathcal{A}(s)$.

5.2.3 Policy

The **policy** tells the agent which actions to take at every state. The policy can be deterministic or stochastic. We denote the probability of taking action a under state s following policy π as $\pi(a|s)$.

5.2.4 Reward

Reward is one of the most unique concept of reinforcement learning. After the execution of action, the agent receives a reward, denoted as r. The agent's goal is to maximize the total reward that the agent receives. Due to this goal, the reward can encourage or discourage the agent's choice of selecting an action under certain state. Again, the reward can be stochastic, too.

One question that follows naturally is: Can't we just pick the actions that gives the most reward at every state? If this was correct, the reinforcement learning may not have existed. Since the reward is just an immediate information of the act, choosing only the action with highest reward is not the correct answer, most of the time. There can be a better choice in the long run.

5.3 Trajectories, Returns and Episodes

5.3.1 Trajectory

The **trajectory** is the footsteps of the agent. More formally, suppose that the agent is on the initial state s_0 and took action a_0. The agent receives reward r_1 from environment and moves to state s_1. Again, the agent take an action a_1, receive reward r_1 and move to state s_2. Repeating this, we get the trajectory of an agent:

$$s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_{T-1}, a_{T-1}, rT. \tag{5.2}$$

5.3.2 Returns

Given the trajectory $\tau = \{s_0, a_0, r_1, s_1, a_1, r_2, \ldots, s_{T-1}, a_{T-1}, rT\}$, we can calculate the **return** G_t, of the trajectory. The naive definition of the return, called **total rewards** is the sum

$$G_t = \sum_{k=t+1}^{T} r_k. \tag{5.3}$$

The point is that we do not know if the terminal time T is finite or not. If T is not finite, than the cummulative reward 5.3 diverges. For this, we introduce the **discounted return**, defined as

$$G_t = \sum_{k=t+1}^{\infty} \gamma^{k-t-1} r_k \tag{5.4}$$

where $\gamma \in [0, 1]$ is called **discount rate**. This works for for finite trajectory too, if we let $r_k = 0$ for $k > T$.

5.3.3 Episodes

The **episode** is the finite trajectory itself, like 5.2. The episode may stop at the states called **terminal states**. The tasks with episodes are called **episodic tasks**. Unlike episodic tasks, there are **continuing tasks**, which does not ends. These two can be expressed in unified mathematical manner. For this, in the episodic task, we must consider the agent's behavior after the terminal state: to stay in that state forever, or to take action like normal states. In this note, we follow the latter, like [Zha25].

5.4 Markov Decision Process

5.4.1 Definition

The **Markov decision process** (MDP) is the general framework that describes the stochastic dynamic systems. It is defined as following:

Definition 5.4.1.1 (Markov Decision Process)

The **Markov decision process** is the 5-tuple $(\mathcal{S}, \mathcal{A}, p, \mathcal{R}, \gamma)$, where each elements are defined as following:

- $\mathcal{S}$ is the state space, $\mathcal{A}(s)$ is the action space associated with state s, and $\mathcal{R}(s,a)$ is a set of rewards, associated with each state-action pair (s,a).
- $p(s', r|s, a)$ is the dynamics of the model, which is as probability of moving to state s' from s by taking action a, and obtaining reward r. Note that the state transition probability and the reward probability can be decribed with dynamics. See 5.4.1.1
- $\pi(s|a)$ is the policy, a probability of agent choosing action a in state s.
- γ is the discount rate.

with the dynamics satisfying the *Markov property*:

$$p(s_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) = p(s_{t+1}|s_t, a_t) \tag{5.5}$$

$$p(r_{t+1}|s_t, a_t, s_{t-1}, a_{t-1}, \ldots, s_0, a_0) = p(r_{t+1}|s_t, a_t) \tag{5.6}$$

where t is the current time step.

Remark 5.4.1.1 (Transition and reward probability as dynamics)

The 4-tuple dynamics can be used to describe transition probability and reward probability as following:

$$p(s'|s,a) = \sum_{r \in \mathcal{R}(s,a)} p(s', r|s, a) \tag{5.7}$$

$$p(r|s,a) = \sum_{s' \in \mathcal{S}} p(s', r|s, a) \tag{5.8}$$

The dynamics can be either stationary or non-stationary. If the dynamics changes over time, it is non-stationary; else, it is called stationary.

Chapter 6

Bellman Equation

6.1 Introduction

The **Bellman equation** is the heart of reinforcement learning. All reinforcement learning algorithms are designed to solve this equation.

6.2 Bellman Equation

6.2.1 State-value Function

Before diving into the Bellman equation, we introduce an important concept called the **value of a state**.

Definition 6.2.1.1 (State Value)

Let the agent follows the policy π. The **state value** $v_\pi(s)$ of state s is defined as the expected return:

$$v_\pi(s) = \mathbb{E}[G_t|S_t = s] \tag{6.1}$$

where G_t is the discounted return $G_t = \sum_{k=t+1}^{\infty} \gamma^{k-t-1} R_k$. The function v_π itself is called **state-value function**.

State values are used to evaluate the policy π. Policies that generate greater state values are better. Then, how can we obtain te state values? The answer is to solve the Bellman equation.

6.2.2 Bellman Equation for State-Value Function

The Bellman equation is simply a set of linear equations that describe the relationships of each state values. Bellman equation can be obtained by using the recursive property of return:

$$\begin{aligned} G_t &= R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \gamma^3 R_{t+4} + \cdots && (6.2)\\ &= R_{t+1} + \gamma(R_{t+2} + \gamma R_{t+3} + \gamma^2 R_{t+4}) + \cdots && (6.3)\\ &= R_{t+1} + \gamma G_{t+1}. && (6.4) \end{aligned}$$

Substituting equation 6.1 with the recursive realationship, we obtain

$$v_\pi(s) = \mathbb{E}[G_t|S_t = s] \tag{6.5}$$
$$= \mathbb{E}[R_{t+1} + \gamma G_{t+1}|S_t = s] \tag{6.6}$$
$$= \mathbb{E}[R_{t+1}|S_t = s] + \gamma\mathbb{E}[G_{t+1}|S_t = s] \tag{6.7}$$

Using the law of total expectation 8.2, we get

$$\mathbb{E}[R_{t+1}|S_t = s] = \sum_{a\in\mathcal{A}(s)} \pi(a|s)\mathbb{E}[R_{t+1}|S_t = s, A_t = a] \tag{6.8}$$
$$= \sum_{a\in\mathcal{A}(s)} \pi(a|s) \sum_{r\in\mathcal{R}(s,a)} p(r|s,a)r \tag{6.9}$$

and

$$\mathbb{E}[G_{t+1}|S_t = s] = \sum_{s'\in\mathcal{S}} \mathbb{E}[G_{t+1}|S_{t+1} = s', S_t = s]p(s'|s) \tag{6.10}$$
$$= \sum_{s'\in\mathcal{S}} \mathbb{E}[G_{t+1}|S_{t+1} = s']p(s'|s) \quad \text{(Markov property)} \tag{6.11}$$
$$= \sum_{s'\in\mathcal{S}} v_\pi(s')p(s'|s) \tag{6.12}$$
$$= \sum_{s'\in\mathcal{S}} v_\pi(s') \sum_{a\in\mathcal{A}(s)} p(s'|s,a). \tag{6.13}$$

The final result is given in the following box:

Theorem 6.2.2.1 (Bellman Equation for State-Value Function)

Let π be a policy, and let v_π be a state-value function. Then, the following Bellman equation is satisfied:

$$v_\pi(s) = \sum_{a\in\mathcal{A}(s)} \pi(a|s) \sum_{s'\in\mathcal{S}} \sum_{r\in\mathcal{R}(s,a)} p(s', r|s, a)\big[r + \gamma v_\pi(s')\big] \tag{6.14}$$

6.2.3 Matrix-Vector form of Bellman Equation

We can reformulate Bellman equation in terms of matrix-vector form. For this, we rewrite the Bellman equation given in Theorem 6.2.2.1 as

$$v_\pi(s) = r_\pi(s) + \gamma \sum_{s'\in\mathcal{S}} p_\pi(s'|s)v_\pi(s'), \tag{6.15}$$

where

$$r_\pi(s) = \sum_{a\in\mathcal{A}(s)} \pi(a|s) \sum_{r\in\mathcal{R}(s,a)} p(r|s,a)r \tag{6.16}$$
$$p_\pi(s'|s) = \sum_{a\in\mathcal{A}(s)} \pi(a|s)p(s'|s,a). \tag{6.17}$$

The r_π and p_π denotes the mean of immediate rewards and probability of transitioning from s to s' under policy π, respectively. Suppose that we indexed all states as s_i with $i = 1, \ldots, n$, where $n = |\mathcal{S}|$. Let

$$v_\pi = \begin{bmatrix} v_\pi(s_1) \\ \vdots \\ v_\pi(s_n) \end{bmatrix}, r_\pi = \begin{bmatrix} r_\pi(s_1) \\ \vdots \\ r_\pi(s_n) \end{bmatrix}, P_\pi = \begin{bmatrix} p_\pi(s_1|s_1) & p_\pi(s_2|s_1) & \ldots & p_\pi(s_n|s_1) \\ p_\pi(s_1|s_2) & p_\pi(s_2|s_2) & \ldots & p_\pi(s_n|s_2) \\ \vdots & \vdots & & \vdots \\ p_\pi(s_1|s_n) & p_\pi(s_2|s_n) & \ldots & p_\pi(s_n|s_n) \end{bmatrix}. \tag{6.18}$$

Then, Theorem 6.2.2.1 can be written in the following form:

Theorem 6.2.3.1 (Matrix-Vector form of Bellman Equation)

Let v_π, r_π and P_π be given as 6.18. Then, following is satisfied:

$$v_\pi = r_\pi + \gamma P_\pi v_\pi \tag{6.19}$$

Remark 6.2.3.1 (Properties of Matrix P_π)

Note that the matrix P_π given in equation 6.18 satisfies following properties:

- All the elements are nonnegative;
- The sum of the values of every row equals 1. Matrix with this property is called a *stochastic matrix*.

6.3 Solving The Bellman Equation

There are two methods in solving the Bellman equation. The closed-form solution, and the iterative solution.

6.3.1 Closed-form Solution

Since we have represented the Bellman equation as the matrix form, solving it is very easy.

Theorem 6.3.1.1 (Closed-form Solution of The Bellman Equation)

Let π be a given policy, and v_π, r_π, P_π be given as 6.18. The solution to the equation $v_\pi = r_\pi + \gamma P_\pi v_\pi$ is given as

$$v_\pi = (I - \gamma P_\pi)^{-1} r_\pi \tag{6.20}$$

Proof. Manipulate the equation as following:

$$v_\pi = r_\pi + \gamma P_\pi v_\pi \tag{6.21}$$
$$v_\pi - \gamma P_\pi v_\pi = r_\pi \tag{6.22}$$
$$(I - \gamma P_\pi) v_\pi = r_\pi. \tag{6.23}$$

Now, what we have to show is that the matrix $I - \gamma P_\pi$ is invertible. We use the fact that A is invertible $\Leftrightarrow$ eigenvalues of A are nonzero. To show this, we use Geshgorin Circle Theorem 9.2.0.1. Note that the i-th Geshgorin circles has a center at $1 - \gamma p(s_i|s_i)$, with radius $\sum_{j \neq i} \gamma p_\pi(s_j|s_i)$. Since P_π is stochastic matrix, we know that $\sum_j \gamma p(s_j|s_i) = \gamma < 1$. That is,

$\sum_{j \neq i} \gamma p_\pi(s_j|s_i) < 1 - \gamma p_\pi(s_i|s_i)$, which means that the i-th Geshgorin circle cannot contain the origin. According to the Geshgorin Circle Theorem, all the eigenvalues cannot be zero. □

TODO: add properties of $I - \gamma P_\pi$.

6.3.2 Iterative Solution

The direct method for solving the Bellman equation is not used very well in practice, since the inverse matrix computation is very costly. Instead of the close-form method, in practice, we use iterative method to solve the Bellman equation.

Theorem 6.3.2.1 (Iterative Solution of The Bellman Equation)

Let π be a policy and v_π, r_π, and P_π be that of 6.18. Then, the solution of 6.2.3.1 equals the limit of following sequence $\{v_k\}$:

$$v_{k+1} = r_\pi + \gamma P_\pi v_k \tag{6.24}$$

Proof. What we have to show is that $v_k \to v_\pi$ as $k \to \infty$. For this, we let $\delta_k = v_k - v_\pi$ and show that $\delta_k \to 0$. Substituting $v_{k+1} = \delta_{k+1} + v_\pi$ and $v_k = \delta_k + v_\pi$ into equation 6.24, we have

$$\delta_{k+1} = \gamma P_\pi \delta_k. \tag{6.25}$$

Since P_π is a stochastic matrix and $0 < \gamma < 1$, we know that $\delta_k \to 0$ as $k \to \infty$. □

6.4 Bellman Equation for Action-Value Function

Now we introduce the **action value**, which denotes the value of an action at a state.

6.4.1 Action Value

The action value is defined as following:

Definition 6.4.1.1 (Action Value)

Let π be a given policy. Then, the **action value** of a state-action pair is defined as

$$q_\pi(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]. \tag{6.26}$$

Remark 6.4.1.1 (Properties of Action Value)

Action value $q_\pi(s, a)$ has following properties.

- $v_\pi = \sum_{a \in \mathcal{A}(s)} \pi(a|s) q_\pi(s, a)$
- $q_\pi(s, a) = \sum_{r \in \mathcal{R}(s,a)} p(r|s, a) r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) v_\pi(s')$

Proof.
- Use the law of total expectation 8.2.

- Use the result above and 6.2.2.1.

□

6.4.2 Bellman Equation for Action-Value Function

The Bellman equation for action-value function is already in our hand, since actually the Remark 6.4.1.1 is what all we need.

Theorem 6.4.2.1 (Bellman Equation for Action-value Function)

Let π be a given policy, and let $q_\pi(s,a)$ be an action value. Then, the following Bellman equation is satisfied:

$$q_\pi(s,a) = \sum_{r\in\mathcal{R}(s,a)} p(r|s,a)r + \gamma \sum_{s'\in\mathcal{S}} p(s'|s,a) \sum_{a'\in\mathcal{A}(s')} \pi(a'|s')q_\pi(s',a') \tag{6.27}$$

Moreover, the matrix-vector form is given as

$$q_\pi = \tilde{r} + \gamma \tilde{P}\Pi q_\pi \tag{6.28}$$

where $[q_\pi]_{(s,a)} = q_\pi(s,a)$, $\tilde{r}_{(s,a)} = \sum_{r\in\mathcal{R}(s,a)} p(r|s,a)r$, $[\tilde{P}]_{(s,a),s'} = p(s'|s,a)$ and Π is a block diagonal matrix such that $\Pi_{s',(s',a')} = \pi(s'|a')$ for $a' \in \mathcal{A}(s')$ and other entries are all zero.

Proof. Use the Remark 6.4.1.1. For the matrix-vector form, try to derive it, as we did in Theorem 6.2.3.1. □

Chapter 7

Bellman Optimal Equation

7.1 Introduction

This chapter introduces the Bellman optimal equation (BOE), and proves serveral properties of it. This section mainly follows the book [Zha25].

7.2 Optimal Policy

The reinforcement learning algorithms are all designed to find an optimal policy for given enviroment. Then, what is an 'optimal' policy? The definition is given below.

Definition 7.2.0.1 (Optimal Policy and Optimal State Value)

A policy π^* is optimal if and only if $v_\pi^*(s) \geq v_\pi(s)$ for all $s \in \mathcal{S}$ and for any other policy π. The state values of π^* are the optimal state values.

Definition 7.2.0.2 (Better Policy)

A policy π_1 is **better** than a policy π_2 if and only if $v_{\pi_1}(s) \geq v_{\pi_2}(s)$ for all $s \in \mathcal{S}$.

The questions that naturally arise are:

- Existence: Does the optimal policy exists?
- Uniqueness: Is the optimal policy unique?
- Stochasticity: Is the optimal policy stochastic or deterministic?
- Algorithm: How do we obtain the optimal policy and the optimal state values?

7.3 Bellman Optimality Equation

The optimal policy and optimal state values can be analysed via the **Bellman optimality equation** (BOE). The optimal state values satisfies following Bellman optimality equation. This claim in fact has two logically independent statements:

1. The BOE, viewed purely as a fixed-point equation, has a unique solution (Section 7.4);
2. The unique solution of BOE concides with the optimal stat-value function v^*, and the derived greedy policy is optimal (Section 7.5)

Note that the Section 7.4 does not assume that the state-value function is optimal.

Definition 7.3.0.1 (Bellman Optimality Equation)

Let Π be a set of possible policies and let $\gamma \in (0,1)$ be a discount rate. Then, the Bellman Optimality Equation is defined as

$$v(s) = \max_{\pi \in \Pi} \sum_{a \in \mathcal{A}(s)} \pi(a|s) \left(\sum_{r \in \mathcal{R}} p(r|s,a) r + \gamma \sum_{s' \in \mathcal{S}} p(s'|s,a) v(s') \right) \tag{7.1}$$

$$= \max_{\pi \in \Pi} \sum_{a \in \mathcal{A}(s)} \pi(a|s) q(s,a) \tag{7.2}$$

where $v(s), v(s')$ are unkown variables to be solved. Additionally, the matrix-vector form of Bellman optimality equation is given as

$$v = \max_{\pi \in \Pi} (r_\pi + \gamma P_\pi v) \tag{7.3}$$

where r_π and P_π are given as 6.18.

Using the matrix-vector form of BOE, we can express BOE very simply as following:

$$v = f(v) \tag{7.4}$$

where $f(v) = \max_{\pi \in \Pi}(r_\pi + \gamma P_\pi v)$.

7.4 Solving The Bellman Optimality Equation

7.4.1 Contraction Mapping Theorem

We propose a **contraction mapping theorem**, the main tool for solving the BOE. The contraction mapping theorem is a powerful tool for analyzing general nonlinear equations.

Definition 7.4.1.1 (Contraction Mapping)

Let $f : \mathbb{R}^d \to \mathbb{R}^d$. Then, the function f is a **contraction mapping** if there exists $\gamma \in (0,1)$ such that

$$\|f(x_1) - f(x_2)\| \leq \gamma \|x_1 - x_2\| \tag{7.5}$$

for all $x_1, x_2 \in \mathbb{R}^d$.

Now we present the contraction mapping theorem.

Theorem 7.4.1.1 (Contraction Mapping Theorem)

Let $f : \mathbb{R}^n \to \mathbb{R}^n$ be a contraction mapping. Then, the following properties hold for equation $x = f(x)$:

- Existence: There exists a fixed point x^* satisfying $f(x^*) = x^*$;
- Uniqueness: The fixed point x^* is unique;
- Algorithm: The following sequence $\{x_k\}$ converges to the fixed point x^* exponentially, for any initial x_0.

$$x_{k+1} = f(x_k) \tag{7.6}$$

Proof. □

7.4.2 Contraction Property of Bellman Optimality Equation

Now, to use the contraction mapping theorem, we prove that the right-hand side of BOE 7.3 is a contraction mapping.

Theorem 7.4.2.1 (Contraction Property of Bellman Optimality Equation)

Let the function f be that of equation 7.3. Then, the following contraction mapping property holds for any $v_1, v_2 \in \mathbb{R}^{|\mathcal{S}|}$:

$$\|f(v_1) - f(v_2)\|_\infty \leq \gamma \|v_1 - v_2\|_\infty \tag{7.7}$$

where $\gamma \in (0, 1)$ is the discount rate, and $\|\|_\infty$ is the maximum norm, which is the maximum absolute value of the elements in a vector.

Proof. □

7.5 Optimal Policy, Optimal State Value and Bellman Optimality Equation

Now, we prove that the solution of the Bellman optimality equation is really an optimal state-value function, and prove that the greedy policy derived from it is the optimal policy.

Part III

Appendices

Chapter 8

Probability Theory

8.1 Introduction

A short, useful theorems of probability theory.

8.2 Law of Total Expectation

The followinga are the law of total expectations.

$$\mathbb{E}[X] = \sum_{a} \mathbb{E}[X|A = a]p(a) \tag{8.1}$$

$$\mathbb{E}[X|A = a] = \sum_{b} \mathbb{E}[X|A = a, B = b]p(b|a) \tag{8.2}$$

Chapter 9

Linear Algebra

9.1 Introduction

A short useful theorems of linear algebra.

9.2 Geshgorin Circle Theorem

Theorem 9.2.0.1 (Geshgorin Circle Theorem)

Bibliography

[Gra20] Laura Graesser. *Foundations of deep reinforcement learning. Theory and practice in Python*. Ed. by Wah Loon Keng. Addison Wesley data & analytics series. Includes bibliographical references and index. Boston: Addison-Wesley, 2020. 379 pp. ISBN: 0135172381.

[Sut20] Richard S. Sutton. *Reinforcement learning. An introduction*. Ed. by Andrew Barto. Second edition. Adaptive computation and machine learning. Hier auch später erschienene, unveränderte Nachdrucke. Cambridge, Massachusetts: The MIT Press, 2020. 526 pp. ISBN: 9780262039246.

[Zha25] Shiyu Zhao. *Mathematical foundations of reinforcement learning*. [Tsinghua]: Tsinghua University Press, 2025. 1275 pp. ISBN: 9789819739448.